清华 AI 校园助手

thu-ai-assistant

产品方案设计报告

模块二：产品方案设计

清华大学

2026 年春季学期

2026 年 5 月 2 日

目录

1 产品概述 **3**
1.1 目标用户 3
1.2 用户痛点 3
1.3 典型使用场景 4

2 核心功能 **4**
2.1 对话式校园任务入口 4
2.2 统一登录与二次验证 5
2.3 校园能力目录 5
2.4 查询类工具 5
2.5 真实动作与确认机制 6
2.6 体育预约兜底能力 6

3 产品架构与功能图 **6**
3.1 总体架构 6
3.2 后端分层架构 7
3.3 功能结构 7
3.4 产品思维导图 7

4 交互流程 **8**
4.1 登录与二次验证流程 8
4.2 查询类对话流程 9
4.3 真实动作确认流程 10
4.4 体育预约兜底流程 10

5 创新与差异化 **11**
5.1 从功能入口转向任务入口 11
5.2 以工具调用降低大模型幻觉 11
5.3 查询与真实动作分层 12
5.4 能力目录透明化 12
5.5 复杂流程的人机协同兜底 12
5.6 独立 App 形态便于快速验证 12

6 模块二小结 **12**

1 产品概述

Campus Agent（清华校园智能助手）是面向清华师生的对话式校园办事助手。产品以自然语言对话为核心，把课程表、成绩、校园卡、宿舍电费、体育场馆、图书馆、研读间、校内通知等校园能力封装为可由大模型调用的工具。用户只需说出目标，系统自动理解意图、调用工具、整理结果，并在充值、预约、取消等真实操作前要求用户确认。

当前实现由两个主要工程组成:移动端 `thu-agent-app` 和后端 `thu-ai-assistant`。移动端负责登录、对话、能力快捷入口、结果渲染和动作链接展示；后端负责会话管理、二次验证、工具调用循环、校园数据适配、待确认动作和体育预约浏览器自动化。底层校园系统能力复用 `@thu-info/lib`，从而与现有 `thu-info-app` 的数据能力保持一致。

1.1 目标用户

产品的核心用户是清华大学在校学生，同时兼顾研究生、助教和需要频繁处理校园事务的校内工作人员。表 1 展示了主要用户画像。

表 1: 目标用户画像

用户类型	典型需求	当前痛点	产品价值
本科生	查课表、查成绩、查体育场地、查校园卡余额	校园系统入口多，查询路径长，信息分散	用一句话完成跨系统查询，结果直接结构化展示
研究生	查收入、查研读间、查图书馆资源、查通知	研读间、财务、通知系统分散，操作频率高	将常用流程收敛到统一对话入口
住宿学生	查电费、查宿舍卫生成绩、查校园网设备	生活服务散落在不同页面，移动端操作繁琐	快速查询余额、记录和设备状态
重度校园 App 用户	同时使用课表、图书馆、体育、校园卡等功能	需要记住每个功能在哪、怎么填参数	Agent 根据语义补齐流程，减少点击和记忆成本

1.2 用户痛点

产品要解决的核心痛点如下:

1. **入口碎片化**。课表、成绩、校园卡、体育场馆、图书馆、校园网等功能分布在不同系统或不同页面中，用户需要知道“去哪里查”。

2. **操作链路长**。体育场馆预约、图书馆座位查询、研读间预约等任务需要多次选择日期、场馆、楼层、区域和时段。

3. **语义门槛高**。用户自然表达通常是“明天羽毛球还有空位吗”“今天下午有什么课”，传统系统则要求用户转换成固定筛选条件。
4. **结果不易读**。原始系统返回格式不统一，用户需要在多个页面之间比较可用时间、费用、状态和限制。
5. **真实动作风险高**。充值、预约、取消预约、设备登录/登出等操作会产生资金、资源占用或账号状态变化，需要明确确认机制。

1.3 典型使用场景

表 2: 典型使用场景

场景	用户表达	系统行为
课表查询	今天下午有什么课?	调用课表工具，按星期和节次整理课程、地点与当前周次
体育余量查询	明天羽毛球场有没有空位?	匹配羽毛球相关场馆，查询日期余量，按场馆、场地、时段展示
校园卡查询	查一下我的校园卡余额	调用校园卡工具，返回余额、状态和过渡余额
校园卡充值	校园卡充 50 元	创建待确认动作，用户确认后才生成支付订单并返回二维码动作
图书馆座位	查北馆今天有哪些空座	按图书馆、楼层、区域逐步查询座位余量
研读间预约	帮我预约明天 10:00–11:00 的研读间	查询可用资源，生成待确认动作，确认后提交预约
校内通知	查一下最新校内通知	查询新闻通知列表，按标题、来源、日期结构化展示

2 核心功能

2.1 对话式校园任务入口

`thu-agent-app` 的第一屏即为 Agent 对话，而不是传统功能宫格。用户输入自然语言后，App 将消息发送到 `/api/chat`，由 `thu-ai-assistant` 判断意图、调用工具并生成结构化中文回复。

当前移动端已实现后端地址配置、健康检查、登录态展示、二次验证输入、校园任务对话、快捷提示、Markdown 结果渲染、支付和预约动作链接渲染。Android 模拟器默认连接 `http://10.0.2.2:3000`，真机调试时可填写电脑局域网 IP。

2.2 统一登录与二次验证

后端通过 `session-manager.ts` 管理 `InfoHelper` 实例和登录态。用户登录后，同一会话内所有工具复用已认证的 `InfoHelper`，避免每次查询重复登录。登录流程包括创建 `sessionId`、启动 `helper.login()`、选择微信/短信/TOTP 二次验证、提交验证码、固定设备指纹、会话超时和登出。会话默认 2 小时超时。

2.3 校园能力目录

后端通过 `/api/capabilities` 暴露能力目录。该目录在产品侧不强调底层工具接口，而是用于告诉用户“现在可以稳定使用什么、哪些正在接入、哪些会在后续版本完善”。因此，能力目录更适合按版本成熟度划分，而不是逐项列出函数名称。表 3 给出了当前产品能力规划。

表 3: 校园能力分阶段规划

阶段	能力定位	覆盖内容	产品说明
已经完成	可稳定演示与使用的核心查询能力	课表查询、成绩与绩点查询、教学日历、教室查询、校园卡余额、宿舍电费余额、校内新闻通知、体育场馆余量查询、基础图书馆信息查询	覆盖用户最高频的“查信息”场景，数据来自真实校园系统；Agent 负责理解意图、自动取数和结构化展示，是当前 MVP 的主要验收范围。
正在集成	已有底层能力或部分流程，正在补齐 Agent 闭环的校园事务能力	校园卡充值、电费充值、体育场馆预约与取消、图书馆座位预约、研读间预约、校园网账号与在线设备、财务收入与发票、体测成绩、宿舍卫生成绩、选课信息与培养方案查询	这部分能力已经具备不同程度的数据接口或流程验证，但涉及支付、预约、取消、验证码、敏感账号操作等真实动作，需要继续完善确认协议、异常处理和前端动作渲染。
计划完善	面向完整校园 Agent 的后续扩展能力	选课/退课等强事务操作、评教提交、新闻订阅与收藏管理、饮水和洗衣等第三方生活服务、校园地图与路线、主动提醒、文档问答和个性化推荐	这些能力需要更强的安全控制、权限边界、第三方系统适配或知识库支撑。后续将在稳定查询能力和确认协议成熟后逐步开放。

2.4 查询类工具

查询类工具的目标是“自动取数 + 结构化表达”。模型不直接编造结果，而是通过工具拿到后端 JSON，再按系统提示词规定的表格、列表或信息卡格式输出。课程表、

成绩、体育场馆、校园卡、电费、图书馆、新闻、教室和校园网均属于这一类。

2.5 真实动作与确认机制

真实动作包括充值、预约、取消预约、支付、设备登录/登出等。产品采用“准备动作 → 用户确认 → 执行动作”的统一协议：

1. 用户提出真实动作，例如“校园卡充 50 元”。
2. Agent 调用对应 prepare 工具，例如 `recharge_campus_card`。
3. 工具创建 `PendingAction`，返回 `confirmation_token`、摘要、风险等级和过期时间。
4. Agent 向用户复述动作摘要和风险，请用户确认。
5. 用户明确回复“确认/执行/充值/确定”等表达。
6. `/api/chat` 检测确认意图后调用 `confirm_pending_action`。
7. 后端执行真实动作，并通过 `actions` 字段把支付二维码、打开链接或验证码面板交给前端渲染。

该机制解决了 Agent 产品中最关键的安全问题：模型可以帮助用户准备操作，但不能在用户未确认时直接产生真实后果。

2.6 体育预约兜底能力

体育预约是当前实现中最复杂的场景。系统同时提供 API 查询路径和 Selenium 交互路径：前者用于查询余量、筛选可预约场地、准备预约参数；后者用于打开真实预约页、复用登录态、处理滑块验证码和让用户完成最终确认。当直接提交预约遇到验证码或系统限制时，后端可打开真实体育预约页面，由用户在 Chrome 窗口或验证码辅助面板中完成最后一步。

3 产品架构与功能图

3.1 总体架构

图 1 展示了产品从移动端到后端、Agent、工具层和清华校内系统的整体结构。

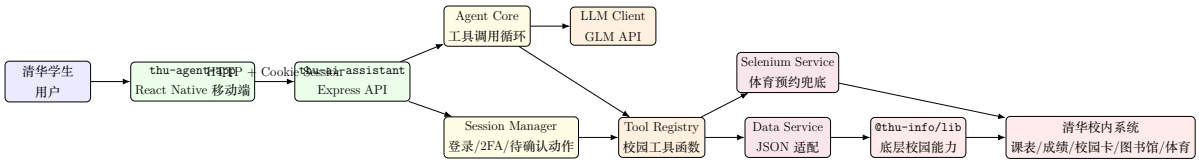

图 1: Campus Agent 总体架构

3.2 后端分层架构

图 2 按工程边界展示后端结构。接口层负责 HTTP API，编排层负责会话和 Agent 循环，工具层负责定义模型可调用函数，服务层负责适配清华系统和浏览器自动化。

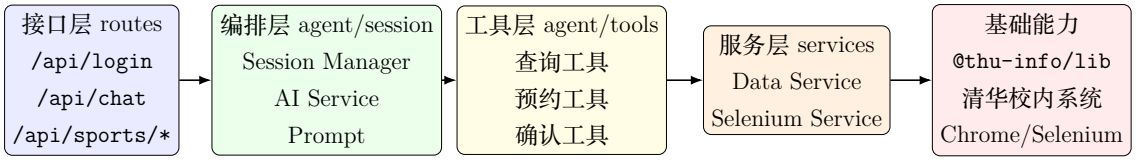

图 2: thu-ai-assistant 后端分层架构

3.3 功能结构

图 3 将产品拆成客户端、后端、校园能力和安全机制四个功能域。

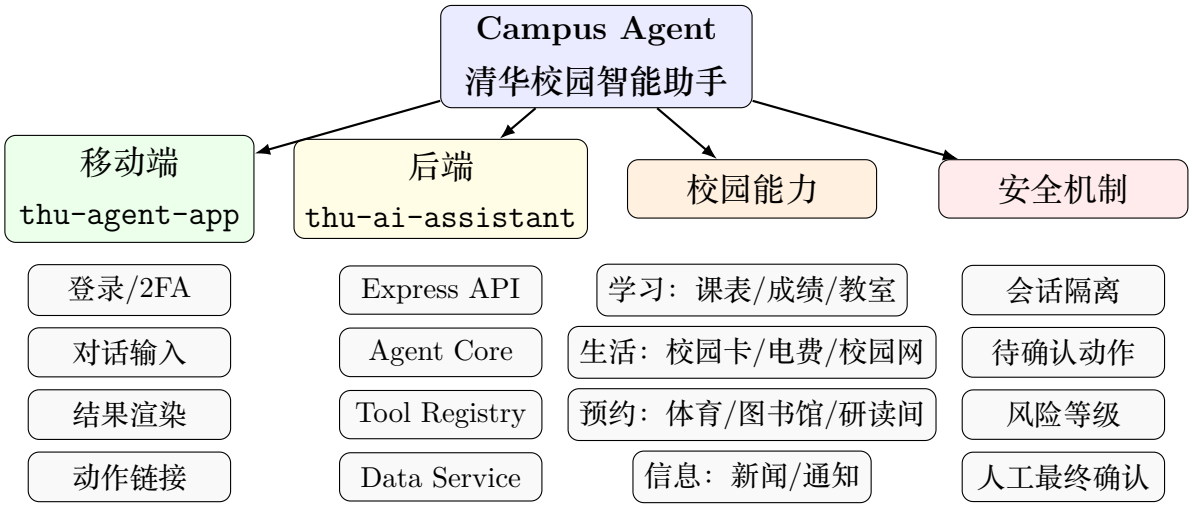

图 3: Campus Agent 功能结构图

3.4 产品思维导图

图 4 用类似 XMind 的横向树状结构概括模块二的产品方案要点，相较放射状思维导图更适合在 A4 报告中阅读。

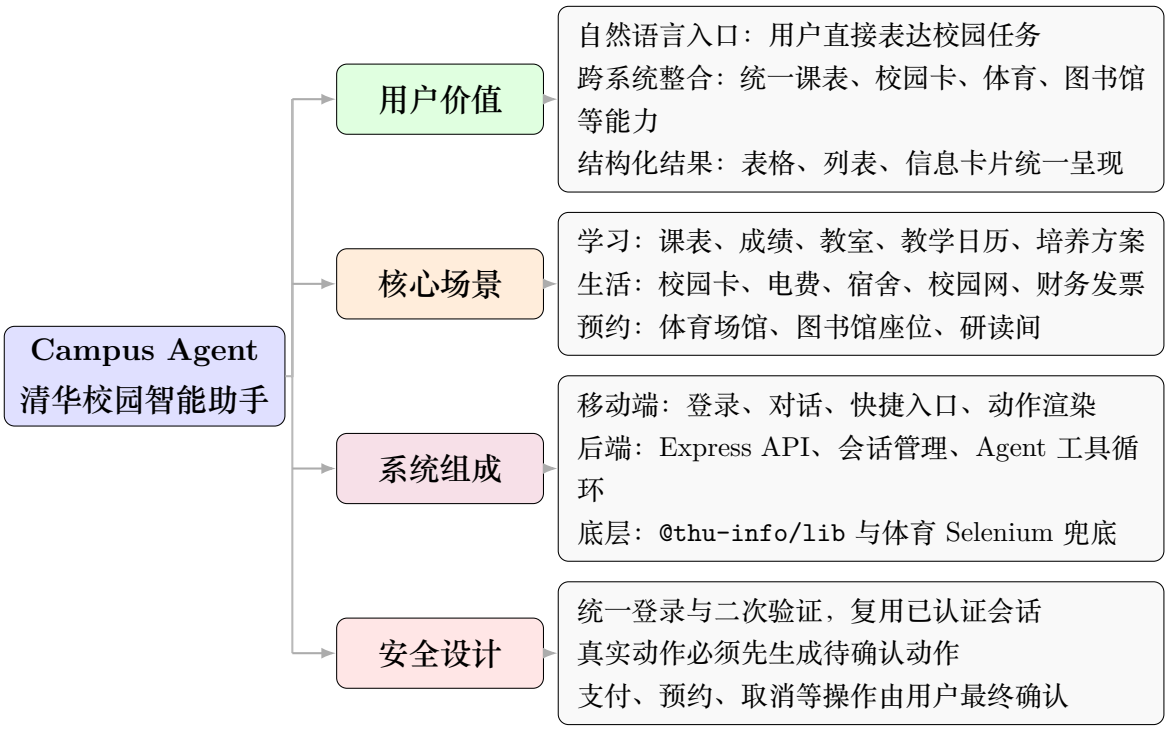

图 4: Campus Agent 产品思维导图（XMind 风格）

4 交互流程

4.1 登录与二次验证流程

用户在移动端输入学号和密码后，后端创建会话并启动 `InfoHelper` 登录。若触发二次验证，前端展示微信、短信或 TOTP 选项并提交验证码；登录完成后，后端将 `loginCompleted` 置为真，后续查询复用同一个认证态。该流程如图 5 所示。

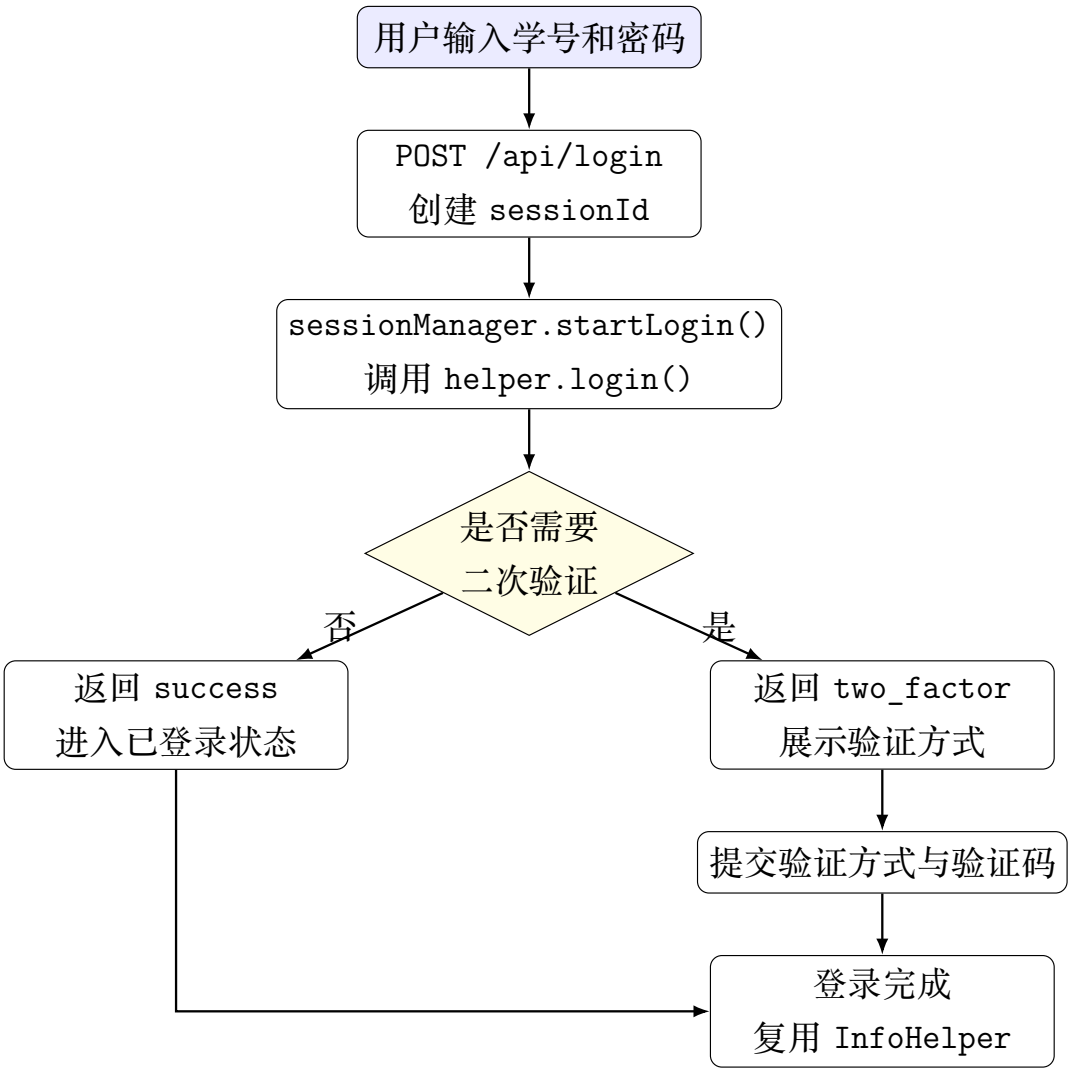

图 5: 登录与二次验证流程

4.2 查询类对话流程

一次典型查询中，App 将用户消息发送到 /api/chat，后端构造消息历史、调用大模型，大模型返回工具调用，后端执行工具并将结果拼回对话，最后由模型生成结构化中文回复。

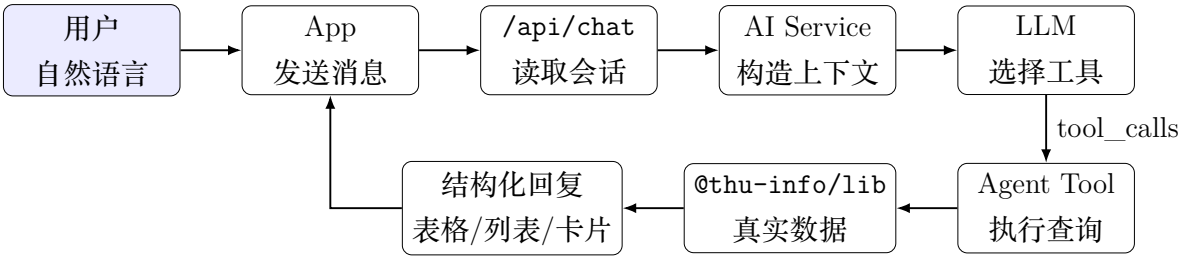

图 6: 查询类对话流程

4.3 真实动作确认流程

图 7 展示了统一确认协议。只有当用户在下一轮明确表达确认意图时，系统才会消费 `confirmation_token` 并执行真实动作。

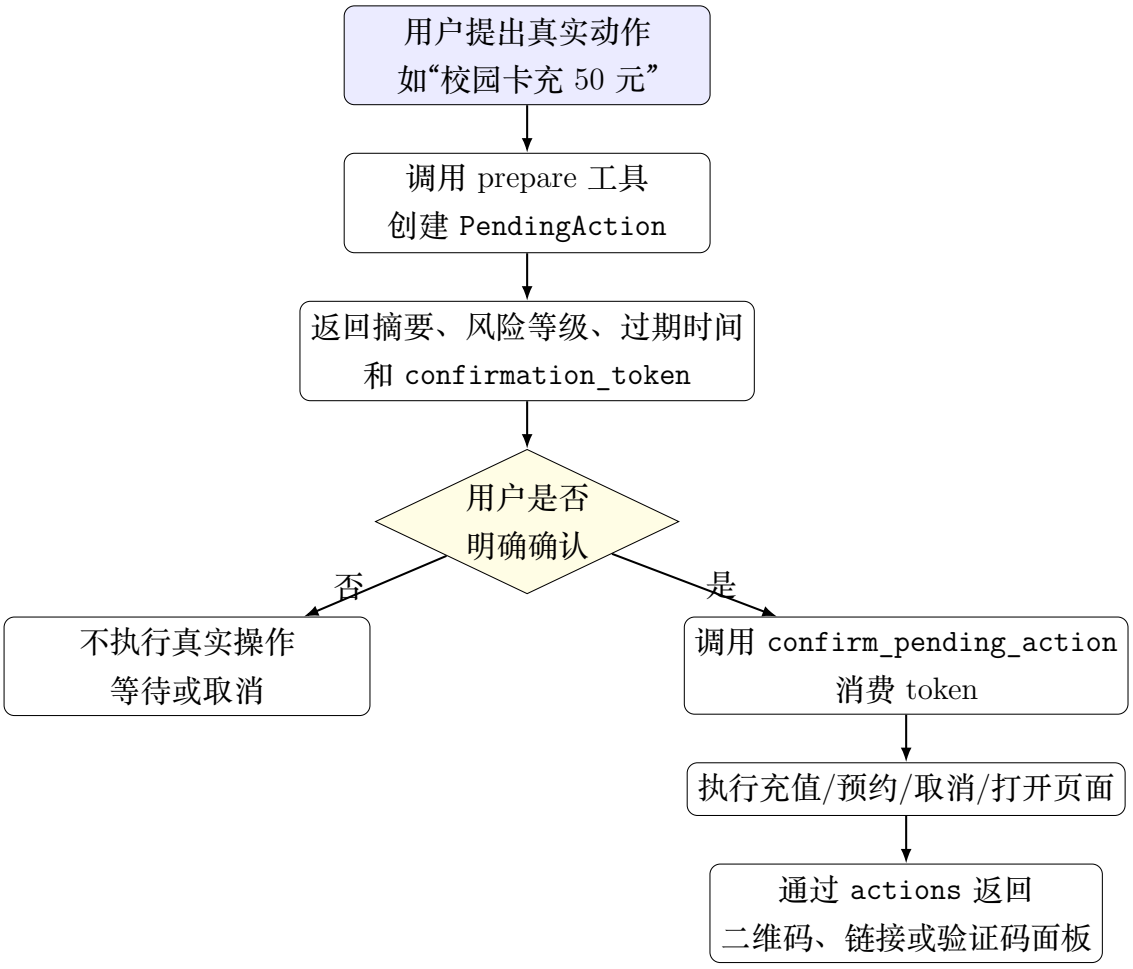

图 7: 真实动作确认流程

4.4 体育预约兜底流程

体育预约流程先查询真实余量并准备预约，提交失败或遇到验证码时，使用 Selenium 打开真实预约页面并交由用户完成最后确认。

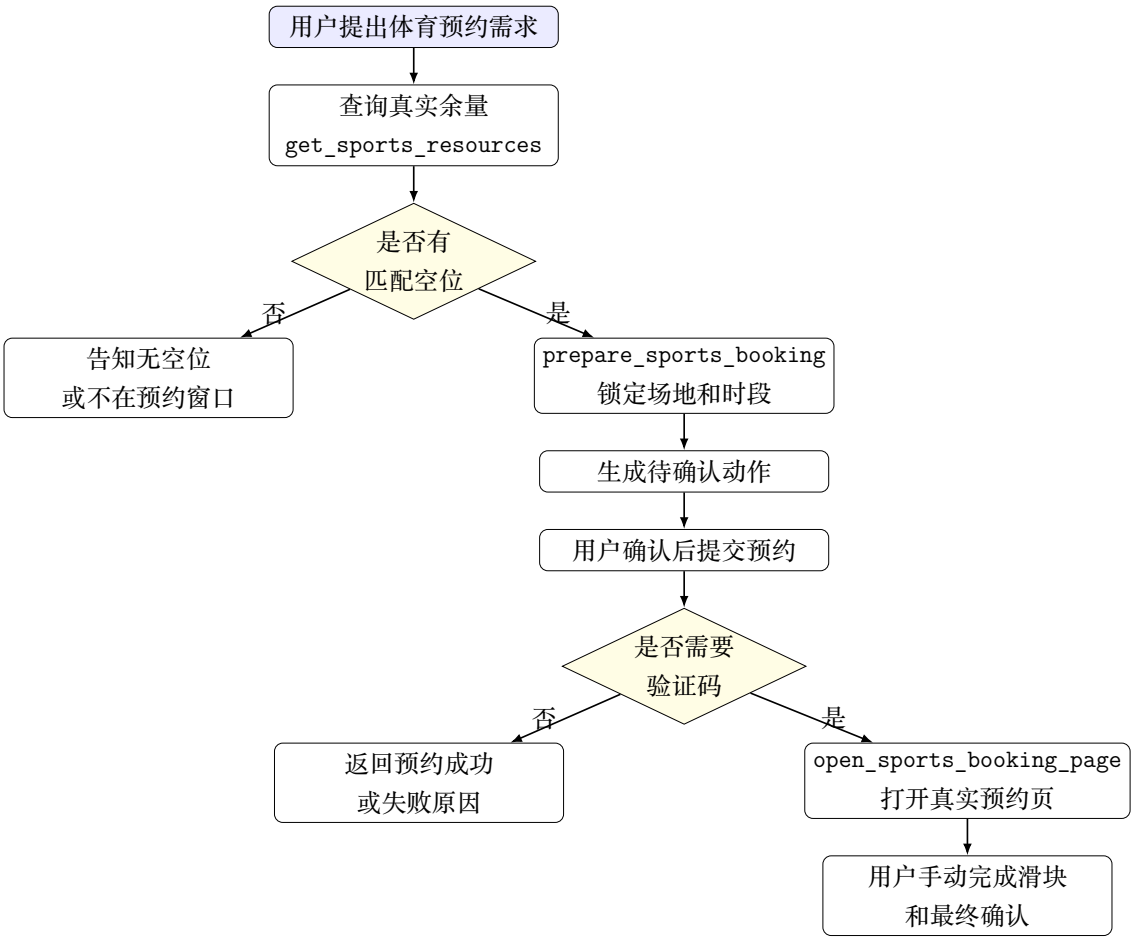

图 8: 体育预约兜底流程

5 创新与差异化

5.1 从功能入口转向任务入口

传统校园 App 的核心是“用户找到功能，再填写表单”。Campus Agent 的核心是“用户表达目标，系统选择工具”。例如用户不需要先进入体育、选择场馆、选择日期，而是直接说“明天羽毛球有没有空位”。系统负责从自然语言中抽取运动类型、日期和查询意图，再调用对应工具。

5.2 以工具调用降低大模型幻觉

系统不是让大模型凭记忆回答校园信息，而是把校园能力拆成明确的工具函数，例如 `get_schedule`、`get_sports_resources`、`get_card_info` 和 `get_library_seats`。模型负责理解意图和组织表达，真实数据来自工具结果。这种设计能显著降低校园信息场景中最危险的“编造余额、编造空位、编造成绩”问题。

5.3 查询与真实动作分层

产品没有把所有能力都做成自动执行。对于查询类任务，系统可以直接返回结果；对于充值、预约、取消等真实动作，必须先生成待确认动作，再等待用户明确确认。这种分层使 Agent 既有办事能力，又不会因为误识别或误触造成资金、预约资源或账号状态变化。

5.4 能力目录透明化

`/api/capabilities` 将校园能力按“已经完成、正在集成、计划完善”三个阶段组织，避免用户误以为所有校园任务都已完全自动化。Agent 在用户询问能力时优先读取能力目录，能够清楚说明哪些能力可以稳定使用，哪些能力正在补齐真实动作闭环，哪些能力属于后续版本规划。

5.5 复杂流程的人机协同兜底

体育预约存在滑块验证码、预约窗口限制、支付或最终确认等复杂情况。产品没有强行宣称全自动，而是设计了 Selenium 打开真实预约页、复用登录态、用户手动完成验证码和最终确认的兜底路径。这种方式更符合校园系统的实际约束，也让产品在 MVP 阶段具备可演示和可扩展的真实链路。

5.6 独立 App 形态便于快速验证

`thu-agent-app` 是独立包名 `com.campusos.agent` 的移动端 Demo，不依赖改造原有 `thu-info-app` 首页。这样可以在不影响原 App 稳定性的前提下快速验证 Agent-first 交互、工具调用协议、真实动作确认机制和结果渲染方式。

6 模块二小结

Campus Agent 的产品方案围绕“自然语言校园任务入口”展开：前端通过独立移动端提供登录、对话和动作渲染；后端通过 Express、Session Manager、Agent 工具注册和 `@thu-info/lib` 把清华校内系统能力统一封装；大模型只负责意图理解、工具选择和结果表达；真实动作统一纳入待确认协议。

该方案的核心优势在于：把分散校园系统整合成一个可对话、可查询、可确认、可扩展的校园 Agent 平台。在当前实现基础上，它已经具备课程表、成绩、校园卡、电费、体育、图书馆、新闻、教室、财务、校园网等多类校园能力的统一入口，并为后续扩展更多真实办事流程留下了清晰架构。